Functional Testing: Designing Tests from the Specification

SE 413/513 — Software Testing

Course Reading Material

Unit: Black-Box (Specification-Based) Test Design

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1 Learning Objectives **3**

2 Where We Are **3**

3 What Is Functional Testing? **3**
3.1 The Test Basis and the Oracle 4

4 From Requirements to Test Cases **4**
4.1 The Running Specification 4
4.2 Test Conditions, Test Cases, and Traceability 4

5 Recap: Equivalence Partitioning and Boundary Values **5**

6 Decision Table Testing **5**
6.1 When to Use It 5
6.2 Step 1: The Full Table 6
6.3 Step 2: Collapse Rules with "Don't Care" Entries 6
6.4 Step 3: One Test per Rule 6
6.5 What It Catches 7

7 State Transition Testing **7**
7.1 When to Use It . . . 7
7.2 The Model . . . 8
7.3 The State Table: Finding Invalid Transitions . . . 8
7.4 Coverage Criteria for State Machines . . . 9
7.5 Implementing the Tests . . . 9

8 Combinatorial and Pairwise Testing **10**
8.1 The Combinatorial Explosion . . . 10
8.2 The Pairwise Idea . . . 11
8.3 Limitations . . . 12

9 Property-Based Testing with Hypothesis **12**
9.1 From Examples to Properties . . . 12
9.2 Fixing the Defect . . . 13
9.3 Finding Good Properties . . . 13

10 Choosing a Technique **14**
10.1 Functional Testing and Coverage Together . . . 14

11 Common Mistakes **14**

12 A Practical Mental Model **15**

13 Summary **15**

14 Practice Exercises **16**

15 Discussion Questions **17**

16 Key Takeaways for SE 413/513 **17**

17 References **17**

1 Learning Objectives

After completing this reading, you should be able to:

- Explain what functional testing is, distinguish its two common meanings, and place it relative to the white-box techniques of Weeks 5–7.
- Turn a written specification into numbered requirements, test conditions, and traceable test cases.
- Build a **decision table** for business rules that combine several conditions, collapse it safely, and derive one test per rule.
- Model stateful behavior as a **state transition** diagram and table, and design tests for all states, all valid transitions, and invalid transitions.
- Explain the combinatorial explosion of input combinations, and use **pairwise testing** (with `allpairspy`) to reduce it.
- Express functional requirements as **properties** and test them with `Hypothesis`.
- Choose an appropriate functional test design technique for a given specification, and implement the resulting tests in `pytest`.

2 Where We Are

Week 3 introduced **specification-based (black-box)** test design: **equivalence partitioning (EP)** and **boundary-value analysis (BVA)**. Weeks 5–7 then turned to **structure-based (white-box)** testing: control-flow coverage, data-flow coverage, and measuring coverage with `coverage.py`.

Week 7 ended with four things a coverage tool cannot see: weak assertions, short-circuited sub-conditions, boundary values, and missing features. Every one of them is a question about *what the program is supposed to do* — which is exactly what functional testing starts from.

Key Idea

White-box testing asks, "Did my tests exercise the code I wrote?" Functional testing asks, "Does the program do what the specification says, for the inputs and situations the specification describes?" A good test strategy needs both.

This reading assumes EP and BVA from Week 3; Section 5 only recaps them briefly. Most of the reading covers four further techniques: decision tables, state transition testing, pairwise (combinatorial) testing, and property-based testing.

3 What Is Functional Testing?

The term *functional testing* is used in two related but different ways, and it is worth keeping them apart[2, 3]:

1. **A test design approach.** Tests are derived from the *function* the software is specified to compute — its inputs, outputs, and rules — without reference to the implementation. In this sense "functional testing" is a synonym for **specification-based** or **black-box** testing[3].

2. **A test type.** Tests that evaluate *what* the system does (its functional suitability), as opposed to **non-functional** testing of *how well* it does it: performance, security, usability, reliability[2].

This reading uses the first meaning: systematic techniques for choosing test inputs and expected results *from a specification*. These techniques apply at every test level — a single function (unit), a set of modules (integration, Week 9), or the whole system (Weeks 10–11).

3.1 The Test Basis and the Oracle

Two terms recur throughout:

- The **test basis** is whatever describes the intended behavior: requirements, user stories, API documentation, business rules, a docstring.
- The **test oracle** is the source of the *expected result* for each test. In functional testing, the oracle is the specification — never the current behavior of the code. A test whose expected value was copied from what the code happened to return only confirms that the code does what it does.

Common Pitfall

Writing expected values by running the code. If you run `shipping_fee(20.0, True, True)`, see `10.0`, and paste `10.0` into a test, that test can never detect a defect in that case. Derive every expected value from the specification first, then run the code.

4 From Requirements to Test Cases

4.1 The Running Specification

This reading extends the checkout example from Week 7 with an `express` shipping option and an order lifecycle. The specification, as a product owner might write it:

Checkout specification (excerpt)

REQ-1 Express shipping costs \$10 for members and \$15 for everyone else, regardless of order amount.
REQ-2 Standard shipping is free for members.
REQ-3 Standard shipping is free for any order of \$50 or more.
REQ-4 Otherwise, standard shipping costs \$7.99.
REQ-5 An order is created, then paid, then shipped, then delivered, in that order.
REQ-6 An order can be cancelled before it ships. Cancelling a paid order refunds the payment.
REQ-7 Any other action on an order is rejected and leaves the order unchanged.
REQ-8 A bill split among n people produces n shares, in cents, that add up exactly to the total.

Numbering the requirements is the first step. It lets every test say *which requirement it checks*, and lets you see which requirements have no test at all.

4.2 Test Conditions, Test Cases, and Traceability

A **test condition** is a testable aspect of the test basis ("express shipping for a member"); a **test case** is a concrete set of inputs, preconditions, and an expected result that exercises it ("`shipping_fee(20.0, True, True)` returns `10.0`")[2]. Recording which test cases cover which requirements gives a **traceability matrix**:

Req.	Technique used	Tests (this reading)
REQ-1–4	Decision table + BVA	`test_shipping_decision_table[R1..R5]`, `test_free_shipping_boundary`
REQ-5–7	State transition testing	`test_valid_transitions`, `test_invalid_transitions_rejected`, refund tests
REQ-8	Property-based testing	`test_shares_add_up_to_total`

Tip

In `pytest`, a cheap and effective form of traceability is to put the requirement or rule ID in the test name or in a `pytest.param(..., id="R1-member-express")`. The ID then appears in every test report and failure message.

5 Recap: Equivalence Partitioning and Boundary Values

The first functional techniques to reach for are still EP and BVA (Week 3). Applied to REQ-3 and REQ-4 for a non-member choosing standard shipping, the `amount` input has two valid partitions with a boundary at $50:

Partition	Expected fee	Boundary-value tests
$0 \leq$ `amount` < 50	$7.99 (REQ-4)	`49.99`
`amount` ≥ 50	$0.00 (REQ-3)	`50.0`

```
@pytest.mark.parametrize("amount, expected", [(49.99, 7.99), (50.0, 0.0)])
def test_free_shipping_boundary(amount, expected):
    assert shipping_fee(amount, False, False) == expected
```

This is the test Week 7 showed a 100%-coverage suite was missing. We ran it against an implementation in which `>= 50` had been changed to `> 50`: the `50.0` case failed, as it should.

EP and BVA work one input at a time. The techniques that follow handle what EP and BVA do not: **rules that combine several inputs** (decision tables), **behavior that depends on history** (state transitions), **too many combinations to test** (pairwise), and **requirements that must hold for all inputs** (properties).

6 Decision Table Testing

6.1 When to Use It

Decision tables suit specifications of the form "if *this* and *that*, but not *the other*, then do X": business rules, eligibility checks, pricing, permissions[2, 3]. Their value is that they force you to consider *every combination* of conditions — including combinations the specification forgot to mention.

A decision table has:

- **Conditions** (rows at the top): the inputs to the rule, usually true/false;
- **Actions** (rows at the bottom): the outcomes;

- **Rules** (columns): one per combination of condition values, each with its expected action.

6.2 Step 1: The Full Table

REQ-1–4 involve three conditions: *express?*, *member?*, and *amount* $\geq$ *50?*. Three true/false conditions give $2^3 = 8$ combinations:

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
C1: express?	T	T	T	T	F	F	F	F
C2: member?	T	T	F	F	T	T	F	F
C3: amount $\geq$ 50?	T	F	T	F	T	F	T	F
Fee \$10.00 (REQ-1)	✓	✓						
Fee \$15.00 (REQ-1)			✓	✓				
Fee \$0.00 (REQ-2/3)					✓	✓	✓	
Fee \$7.99 (REQ-4)								✓

Filling in every column is where specification problems surface. Column 5 (member, standard, \$50+) is free under *both* REQ-2 and REQ-3 — consistent, fine. Had REQ-2 said "\$2.99 for members," column 5 would have exposed a *conflict* between two requirements before a single line of code was written.

6.3 Step 2: Collapse Rules with "Don't Care" Entries

When two columns differ in only one condition and have the same action, that condition does not affect the outcome there, and the columns can be merged with a "don't care" entry (—):

	R1	**R2**	**R3**	**R4**	**R5**
C1: express?	T	T	F	F	F
C2: member?	T	F	T	F	F
C3: amount $\geq$ 50?	—	—	—	T	F
Action: fee	\$10.00	\$15.00	\$0.00	\$0.00	\$7.99
Full-table columns	1, 2	3, 4	5, 6	7	8

6.4 Step 3: One Test per Rule

Decision table coverage is the fraction of rules exercised by at least one test. One test per rule gives 100%:

```
import pytest
from shop.shipping import shipping_fee

@pytest.mark.parametrize("amount, is_member, express, expected", [
    pytest.param(20.0, True,  True,  10.0, id="R1-member-express"),
    pytest.param(20.0, False, True,  15.0, id="R2-guest-express"),
    pytest.param(20.0, True,  False,  0.0, id="R3-member-standard"),
    pytest.param(80.0, False, False,  0.0, id="R4-guest-standard-50plus"),
    pytest.param(20.0, False, False, 7.99, id="R5-guest-standard-under50"),
])
def test_shipping_decision_table(amount, is_member, express, expected):
    assert shipping_fee(amount, is_member, express) == expected
```

```
$ pytest tests/test_shipping_dt.py -v
...::test_shipping_decision_table[R1-member-express] PASSED          [ 20%]
...::test_shipping_decision_table[R2-guest-express] PASSED           [ 40%]
...::test_shipping_decision_table[R3-member-standard] PASSED         [ 60%]
...::test_shipping_decision_table[R4-guest-standard-50plus] PASSED   [ 80%]
...::test_shipping_decision_table[R5-guest-standard-under50] PASSED  [100%]
```

6.5 What It Catches

A common implementation mistake with rules like these is getting their *priority* wrong — checking the free-shipping rule before the express rule:

```python
def shipping_fee(amount, is_member, express):    # buggy
    if is_member or amount >= 50:
        return 0.0
    if express:
        return 10.0 if is_member else 15.0
    return 7.99
```

Run against this version, rule R1 fails (a member's express order is free instead of $10):

```
FAILED tests/test_shipping_dt.py::test_shipping_decision_table[R1-member-express]
1 failed, 4 passed
```

Common Pitfall

"Don't care" does not mean "don't test." The same buggy version also makes a *guest's* express order of $80 free — but R2 was tested only at $20, where the bug does not show. A "—" entry is the specification's claim that a condition is irrelevant; the implementation may disagree. When a collapsed rule is cheap to test, test it with the don't-care condition both ways (here: R2 at $20 *and* $80). For high-risk rules, test the full uncollapsed table.

Tip

The decision table and BVA answer different questions. The table decides *which combinations* to test; BVA decides *which values* to use within a rule. R4 and R5 meet at the $50 boundary, so the strongest suite tests them at `50.0` and `49.99` (Section 5).

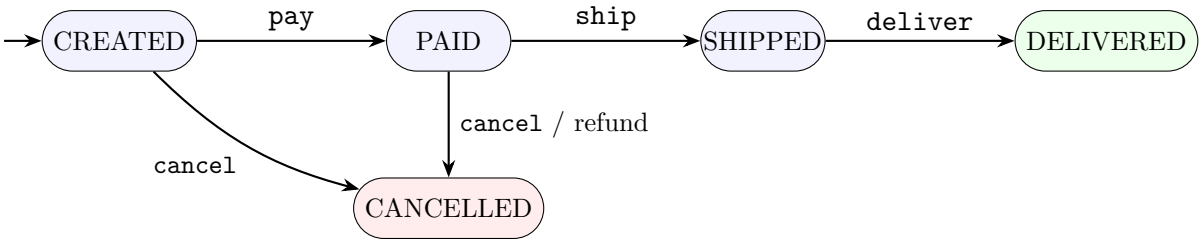
7 State Transition Testing

7.1 When to Use It

Some behavior depends not only on the current input, but on *what has happened before*. Pressing "ship" is correct for a paid order and wrong for a cancelled one. Such systems are modelled as **state machines**: a set of **states**, **events** that occur in each state, and **transitions** that move the system from one state to another[2, 4]. Shopping carts, user sessions, workflow approvals, protocol handlers, and UI wizards are all state machines, whether or not their code says so.

7.2 The Model

REQ-5–7 describe an order lifecycle:



The diagram shows five states, four events, and five valid transitions. An implementation:

```
class InvalidTransition(Exception):
    pass


class Order:
    _TRANSITIONS = {
        ("CREATED", "pay"): "PAID",
        ("CREATED", "cancel"): "CANCELLED",
        ("PAID", "ship"): "SHIPPED",
        ("PAID", "cancel"): "CANCELLED",
        ("SHIPPED", "deliver"): "DELIVERED",
    }

    def __init__(self):
        self.state = "CREATED"
        self.refunded = False

    def _apply(self, event):
        key = (self.state, event)
        if key not in self._TRANSITIONS:
            raise InvalidTransition(f"cannot {event} an order that is {self.state}")
        if key == ("PAID", "cancel"):
            self.refunded = True
        self.state = self._TRANSITIONS[key]

    def pay(self):     self._apply("pay")
    def ship(self):    self._apply("ship")
    def deliver(self): self._apply("deliver")
    def cancel(self):  self._apply("cancel")
```

7.3 The State Table: Finding Invalid Transitions

A diagram shows only what *should* happen. A **state table** lists every (state, event) pair, which makes the *invalid* pairs explicit — and REQ-7 says each of them must be rejected:

State \ Event	`pay`	`ship`	`deliver`	`cancel`
CREATED	PAID	invalid	invalid	CANCELLED
PAID	invalid	SHIPPED	invalid	CANCELLED (+ refund)
SHIPPED	invalid	invalid	DELIVERED	invalid
DELIVERED	invalid	invalid	invalid	invalid
CANCELLED	invalid	invalid	invalid	invalid

5 states × 4 events = 20 pairs: 5 valid and 15 invalid. Invalid transitions are where real defects hide — paying twice, shipping a cancelled order, cancelling after delivery — precisely because they are absent from the "happy path" diagram.

7.4 Coverage Criteria for State Machines

Criterion	Requires that tests…	For `Order`
All states	visit every state at least once	5 states
Valid transitions (0-switch)	exercise every valid transition at least once	5 transitions
All transitions	also attempt every invalid transition and check it is rejected	20 pairs

Valid-transitions coverage subsumes all-states coverage (each state is the target of some transition, except possibly the initial one), and all-transitions coverage is the strongest of the three[2]. "0-switch" refers to sequences of a single transition; "1-switch" coverage would exercise every *pair* of consecutive transitions, and so on — useful when behavior depends on longer histories.

7.5 Implementing the Tests

A helper drives an order into any starting state using only the public API — the tests never set `state` directly, because that would bypass the very transitions under test:

```
import pytest
from shop.order import Order, InvalidTransition

def order_in(state):
    o = Order()
    path = {"CREATED": [], "PAID": ["pay"], "SHIPPED": ["pay", "ship"],
            "DELIVERED": ["pay", "ship", "deliver"],
            "CANCELLED": ["cancel"]}[state]
    for event in path:
        getattr(o, event)()
    return o

@pytest.mark.parametrize("start, event, end", [
    ("CREATED", "pay", "PAID"),
    ("CREATED", "cancel", "CANCELLED"),
    ("PAID", "ship", "SHIPPED"),
    ("PAID", "cancel", "CANCELLED"),
    ("SHIPPED", "deliver", "DELIVERED"),
])
def test_valid_transitions(start, event, end):
    o = order_in(start)
    getattr(o, event)()
```

```
    assert o.state == end

EVENTS = ["pay", "ship", "deliver", "cancel"]
VALID = {("CREATED", "pay"), ("CREATED", "cancel"), ("PAID", "ship"),
         ("PAID", "cancel"), ("SHIPPED", "deliver")}
INVALID = [(s, e)
           for s in ["CREATED", "PAID", "SHIPPED", "DELIVERED", "CANCELLED"]
           for e in EVENTS if (s, e) not in VALID]

@pytest.mark.parametrize("start, event", INVALID)
def test_invalid_transitions_rejected(start, event):
    o = order_in(start)
    with pytest.raises(InvalidTransition):
        getattr(o, event)()
    assert o.state == start          # REQ-7: order unchanged

def test_cancel_after_payment_refunds():
    o = order_in("PAID")
    o.cancel()
    assert o.refunded

def test_cancel_before_payment_does_not_refund():
    o = Order()
    o.cancel()
    assert not o.refunded
```

These 22 tests (5 valid + 15 invalid + 2 refund checks) all pass and give all-transitions coverage of the model. Note the final assertion in `test_invalid_transitions_rejected`: rejecting an event is not enough — REQ-7 also requires that the order be left unchanged, so the test checks both.

Key Idea

The **model** — diagram plus state table — is the test design. Once it exists, the tests are almost mechanical. Most of the thinking (and most of the defects found) happens while building the table and asking, for each empty cell, "what *should* happen here?"

8 Combinatorial and Pairwise Testing

8.1 The Combinatorial Explosion

Consider a checkout page with four independent options:

Parameter	Values
Payment method	card, paypal, gift_card
Delivery	standard, express, pickup
Customer	member, guest
Coupon	none, SAVE10, FLAT5

Testing every combination takes $3 \times 3 \times 2 \times 3 = 54$ end-to-end checkout tests. Add a fifth parameter with four values and it becomes 216. Exhaustive combination testing grows multiplicatively, and quickly becomes impractical for anything slower than a unit test.

8.2 The Pairwise Idea

Pairwise (2-way) testing requires only that *every pair of values* for *every pair of parameters* appear together in at least one test[7]. Because a single test contains many pairs at once (four parameters give six pairs per test), far fewer tests are needed. For our example there are 45 distinct value pairs to cover; a test set covering all of them can be generated with the `allpairspy` library:

```
from allpairspy import AllPairs

params = [
    ["card", "paypal", "gift_card"],
    ["standard", "express", "pickup"],
    ["member", "guest"],
    ["none", "SAVE10", "FLAT5"],
]
for i, row in enumerate(AllPairs(params), 1):
    print(i, list(row))
```

#	Payment	Delivery	Customer	Coupon
1	card	standard	member	none
2	paypal	express	guest	none
3	gift_card	pickup	guest	SAVE10
4	gift_card	express	member	FLAT5
5	paypal	standard	member	SAVE10
6	card	pickup	guest	FLAT5
7	card	express	guest	SAVE10
8	paypal	pickup	member	FLAT5
9	gift_card	standard	guest	FLAT5
10	gift_card	pickup	guest	none

10 tests instead of 54, and we verified with a short script that all 45 value pairs appear. (The theoretical minimum here is 9, since the two 3-valued parameters alone have $3 \times 3 = 9$ pairs; generators trade a test or two for speed.) The rows plug directly into `pytest`:

```
@pytest.mark.parametrize("payment, delivery, customer, coupon",
                         [tuple(r) for r in AllPairs(params)])
def test_checkout_pairwise(payment, delivery, customer, coupon):
    ...
```

Real-World Case Study

How many parameters does it take to trigger a failure? Researchers at the U.S. National Institute of Standards and Technology (NIST) analyzed failure reports from several real systems, including medical-device software and a web browser, and asked how many input parameters had to take specific values for each failure to occur[6]. Most failures were triggered by a single parameter value or by an interaction between two; progressively fewer required three, four, or more; and none of the failures they studied required more than six. This "interaction rule" is the empirical justification for pairwise testing — and for going to 3-way or higher combinations when the stakes are high.

8.3 Limitations

- Pairwise testing does not guarantee finding failures that need three or more specific values at once. For critical features, use a higher strength (t-way) generator such as NIST's ACTS[7].
- It does not choose *values*; EP and BVA still decide which values each parameter should take.
- It assumes parameters are independent. Invalid combinations ("pickup" with a shipping-only payment method, say) must be excluded explicitly; `allpairspy` supports a filter function for this.
- It does not supply expected results. Each generated row still needs an oracle — often from the decision table or the specification.

9 Property-Based Testing with Hypothesis

9.1 From Examples to Properties

Every technique so far picks *specific* example inputs. Some requirements, though, are statements about *all* inputs. REQ-8 is one: for any total and any number of people, the shares must add up exactly to the total. **Property-based testing** turns such a requirement into an executable **property**, and lets a tool generate many inputs to try to falsify it[9, 10].

Consider this first implementation:

```
def split_bill(total, people):
    """Split a total (in dollars) into `people` shares that add up exactly to total."""
    share = round(total / people, 2)
    return [share] * people
```

A few example-based tests pass: `split_bill(30.0, 3)` gives three $10 shares; `split_bill(100.0, 4)` gives four $25 shares. Instead, we state REQ-8 as a property using `Hypothesis`:

```
from hypothesis import given, strategies as st
from shop.split import split_bill

@given(
    total_cents=st.integers(min_value=0, max_value=100_000),
    people=st.integers(min_value=1, max_value=10),
)
def test_shares_add_up_to_total(total_cents, people):
    total = total_cents / 100
    shares = split_bill(total, people)
    assert len(shares) == people
    assert round(sum(shares), 2) == total
```

The `@given` decorator tells Hypothesis to call the test many times (100 by default) with generated values drawn from the **strategies**: here, totals from $0.00 to $1,000.00 in whole cents, and 1 to 10 people. Running it:

```
$ pytest tests/test_split_pbt.py
E       assert 0.02 == 0.01
E        +  where 0.02 = round(0.02, 2)
E        +    where 0.02 = sum([0.01, 0.01])
```

```
E       Failing test case: test_shares_add_up_to_total(
E           total_cents=1,
E           people=2,
E       )
FAILED tests/test_split_pbt.py::test_shares_add_up_to_total - assert 0.02 == ...
```

Hypothesis found that splitting **one cent between two people** produces two one-cent shares — the bill grows by a cent. More generally, any total that does not divide evenly into cents is shared incorrectly (try $100 among 3 people). Note how *small* the reported example is: when Hypothesis finds a failing input, it **shrinks** it, searching for the simplest input that still fails, so the failure is easy to understand.

9.2 Fixing the Defect

Working in integer cents and distributing the remainder one cent at a time satisfies the property:

```python
def split_bill(total, people):
    cents = round(total * 100)
    base, extra = divmod(cents, people)
    return [(base + 1) / 100 if i < extra else base / 100
            for i in range(people)]
```

The fixed version behaves as REQ-8 requires:

```
>>> split_bill(0.01, 2)
[0.01, 0.0]
>>> split_bill(100, 3)
[33.34, 33.33, 33.33]
```

and the property test now passes. Hypothesis also saves failing examples in a local database (the `.hypothesis/` folder) and retries them first on later runs, so a fixed bug that reappears is caught immediately.

9.3 Finding Good Properties

Writing properties is a skill. Common patterns[10]:

Pattern	Example
Invariant	Shares add up to the total; a discount never makes a price negative
Round trip	`decode(encode(x)) == x`; `json.loads(json.dumps(d)) == d`
Idempotence	Normalizing an already-normalized address changes nothing
Comparison with a model	A fast, clever implementation agrees with a slow, obviously correct one
Metamorphic relation	Adding an item never *decreases* a subtotal

Common Pitfall

Re-implementing the function inside the property. A property like `assert split_bill(t, n) == [round(t/n, 2)] * n` copies the implementation's logic — including its bug — and can never fail. A good property states *what must be true of the result*, not *how to compute it*.

Property-based tests complement example-based tests rather than replacing them: examples document specific, important cases (the $50 boundary, each decision-table rule); properties search the space between them for cases nobody thought of.

10 Choosing a Technique

If the specification...	Use	Python support
describes ranges or categories of one input	EP + BVA (Week 3)	`parametrize`
combines several conditions into rules	Decision table	`parametrize` with rule IDs
depends on history or mode	State transition	`parametrize` over (state, event)
has many independent options	Pairwise / t-way	`allpairspy`
states something true for all inputs	Property-based	`Hypothesis`

Real specifications mix these shapes, and so do good test suites: the shipping rules used a decision table *and* BVA; a pairwise checkout suite would draw each row's expected fee from the same decision table.

10.1 Functional Testing and Coverage Together

Week 7's coverage tools are a natural check on a functional suite. Run the specification-based tests under `pytest-cov`: any code they leave unexecuted is either (a) behavior the specification does not mention — ask whether it should exist — or (b) a requirement your tests missed. Coverage tells you *where* to look; the specification tells you *what* the right answer is.

11 Common Mistakes

Common Pitfall

Using the implementation as the oracle. Expected values must come from the specification, not from running the code.

Common Pitfall

Collapsing a decision table and then never testing the don't-care conditions. The implementation may not agree that the condition is irrelevant.

Common Pitfall

Testing only the happy path of a state machine. Most of the 20 (state, event) pairs for `Order` are invalid, and invalid transitions are where defects hide.

Common Pitfall

Setting internal state directly in tests (`o.state = "SHIPPED"`). It bypasses the transitions under test and couples tests to implementation details. Reach states through the public API.

Common Pitfall

Treating pairwise as exhaustive. Pairwise covers all 2-way interactions, not all failures. Raise the strength for high-risk features.

Common Pitfall

Writing properties that restate the code. A property should describe the result, not repeat the algorithm.

12 A Practical Mental Model

When you see "if ... and ... unless ..." in a specification, think:

> "Decision table. How many conditions? Have I filled in every column?"

When the right answer depends on what happened before, think:

> "State machine. What are the states and events — and what should happen in every cell of the state table, not just on the diagram?"

When the number of combinations makes you wince, think:

> "Pairwise first; higher strength where failure is expensive."

When a requirement says "always" or "for any," think:

> "That's a property. Let Hypothesis look for a counterexample."

13 Summary

Technique	Test design unit	Coverage measure
EP / BVA	Partitions and their boundaries	Partitions / boundaries tested
Decision table	Rules (columns)	Rules exercised
State transition	States, transitions, (state, event) pairs	All states $\subset$ valid transitions $\subset$ all transitions
Pairwise	Value pairs across parameters	2-way (or t-way) combinations
Property-based	Properties over generated inputs	— (search, not enumeration)

Term	Meaning
Test basis	The documents or knowledge that describe intended behavior
Test oracle	The source of expected results
Traceability	Mapping between requirements and the tests that check them
Don't-care entry (—)	A condition the specification says does not affect a rule's outcome
0-switch coverage	Every single valid transition exercised
Shrinking	Hypothesis's reduction of a failing input to a minimal example

Key Idea

The specification is the oracle; the technique is the lens. Functional test design is the disciplined conversion of a specification into a model — partitions, a table, a state machine, a parameter set, a property — and of that model into tests.

14 Practice Exercises

Exercise 1 — Decision Table

A course-registration rule says: a student may register if they meet the prerequisites *and* the section has seats *and* they have no account hold. If prerequisites are met and there is no hold but the section is full, the student is waitlisted. In all other cases, registration is rejected. (a) Build the full decision table. (b) Collapse it. (c) Write a parametrized `pytest` test with one case per rule, using rule IDs.

Exercise 2 — Don't-Care Conditions

For your collapsed table from Exercise 1, write an implementation with a plausible bug that one-test-per-rule does *not* detect, then add the test that does.

Exercise 3 — State Transitions

Add a `return_item` event to `Order`: it is valid only from `DELIVERED`, moves the order to a new `RETURNED` state, and refunds the payment. (a) Update the diagram and state table. How many (state, event) pairs are there now, and how many are invalid? (b) Update the tests to keep all-transitions coverage.

Exercise 4 — Pairwise

Add a fifth parameter, *currency* (USD, CAD, EUR, GBP), to the checkout options. (a) How many exhaustive combinations are there? (b) Generate a pairwise set with `allpairspy` and report its size. (c) Exclude the invalid combination "gift_card with a non-USD currency" using `allpairspy`'s filter function.

Exercise 5 — Properties

Write two Hypothesis properties for `apply_coupon(amount, code)` from Week 7 (for non-negative amounts) and run them. Does either find a counterexample? Explain why or why not.

15 Discussion Questions

1. What are the two common meanings of "functional testing," and how do they differ?
2. Why must the test oracle come from the specification rather than the code?
3. How can building a decision table reveal defects in the *specification* before any code exists?
4. What risk does collapsing a decision table introduce, and how do you manage it?
5. Why are invalid transitions often more valuable to test than valid ones?
6. Why should tests reach a state through the public API rather than by setting it directly?
7. What empirical evidence supports pairwise testing, and what does pairwise testing still miss?
8. What makes a good property? Give an example of a property that is useless because it restates the implementation.
9. Why does shrinking matter for a developer debugging a property-test failure?
10. How do functional testing and the coverage analysis of Week 7 complement each other?

16 Key Takeaways for SE 413/513

1. **Functional testing derives tests and expected results from the specification**, never from the implementation.
2. **Number the requirements and trace tests to them** — it shows what is tested and what is not.
3. **Decision tables handle combined conditions;** fill in every column, and be careful with don't-care entries.
4. **State transition testing handles history-dependent behavior;** the state table exposes the invalid transitions where defects hide.
5. **Pairwise testing tames combinatorial explosion,** backed by evidence that most failures involve one or two parameters.
6. **Property-based testing checks "for all" requirements,** and Hypothesis shrinks failures to minimal examples.
7. **Combine techniques, and combine them with coverage:** the specification supplies the right answers, coverage shows where you have not looked.

17 References

- Functional (specification-based) test design techniques: EP, BVA, decision tables, and state transition testing.[2, 3, 4, 5]

- Combinatorial testing and the interaction rule.[6, 7]
- The `allpairspy` library.[8]
- Property-based testing and the Hypothesis library.[9, 10, 11]
- Using `pytest`, this course's default testing framework.[12]
- Course material on software testing principles and related testing practices.[1]

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level (CTFL) Syllabus*, v4.0.1.

[3] Jorgensen, P. C. (2013). *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.

[4] Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.

[5] Aniche, M. (2022). *Effective Software Testing: A Developer's Guide*. Manning Publications.

[6] Kuhn, D. R., Wallace, D. R., & Gallo, A. M. (2004). Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6), 418–421. https://doi.org/10.1109/TSE.2004.24

[7] Kuhn, D. R., Kacker, R. N., & Lei, Y. (2013). *Introduction to Combinatorial Testing*. CRC Press.

[8] Hombashi, T., et al. (2026). *allpairspy: Pairwise test combinations generator*. https://github.com/thombashi/allpairspy

[9] Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 268–279. https://doi.org/10.1145/351240.351266

[10] MacIver, D. R., Hatfield-Dodds, Z., et al. (2019). Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43), 1891. https://doi.org/10.21105/joss.01891

[11] Hypothesis contributors. (2026). *Hypothesis Documentation*. https://hypothesis.readthedocs.io/

[12] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.